

Facade Design Pattern in C# with Examples

In this article, I will discuss the **Facade Design Pattern in C#** with Examples. Please read our previous article discussing the [Adapter Design Pattern in C#](#) with Examples. The Facade Design Pattern falls under the category of **Structural Design Pattern**, i.e., it will deal with Class and Object Composition. As part of this article, we will discuss the following pointers.

1. [What is the Facade Design Pattern in C#?](#)
2. [Understanding the Facade Design Pattern with One Real-Time Example.](#)
3. [Understanding the Class Diagram of Facade Design Pattern.](#)
4. [Implementing Facade Design Pattern in C#.](#)
5. [Advantages of Facade Design Pattern in C#.](#)
6. [When to use the Facade Design Pattern?](#)

What is the Facade Design Pattern in C#?

As per the GOF definition, **Facade Design Pattern states that you need to provide a unified interface to a set of interfaces in a subsystem. The Facade Design Pattern defines a higher-level interface that makes the subsystem easier to use.**

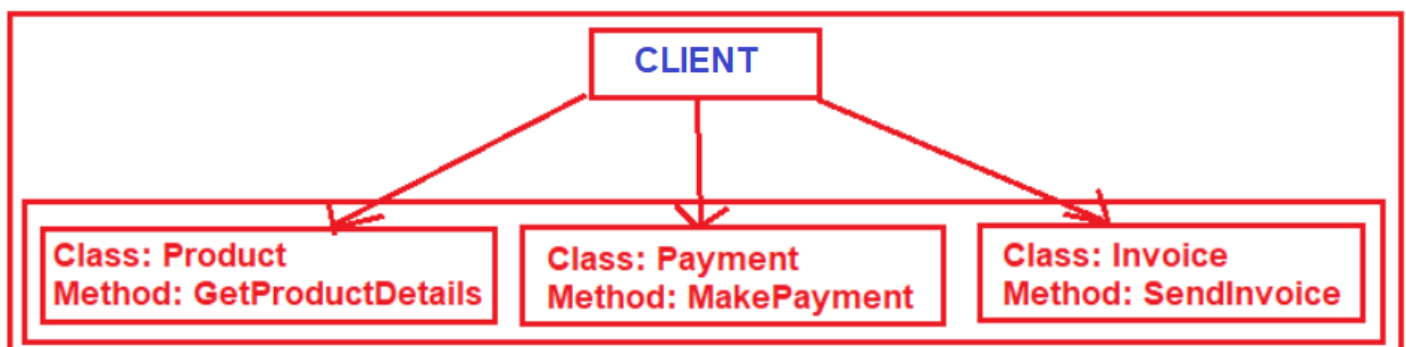
The Facade Design Pattern is a structural pattern that provides a simplified interface to a complex system of classes, libraries, or frameworks. The primary goal of the Facade pattern is to present a clear, simplified, and minimized interface to the external clients while delegating all the complex underlying operations to the appropriate classes within the system. The Facade (usually a wrapper) class sits on the top of a group of subsystems and allows them to communicate in a unified manner.

As the name suggests, Facade means the Face of the Building. Suppose you created one building. The people walking outside the building can only see the walls and glass of the Building. The People do not know anything about the wiring, the pipes, the interiors, and other complexities inside the building. That means the Facade hides all the complexities of the building and displays a friendly face to people walking outside the building.

Understanding Facade Design Pattern in C# with one Real-Time Example:

- **Identify Complex Subsystems:** First, identify the complex parts of your system that need simplification. These could be complex libraries or systems with multiple interacting classes.
- **Create a Facade Class:** Design a facade class that provides a simple interface to the complex subsystems.
- **Delegate Calls to Subsystems:** The facade should delegate the client requests to the appropriate objects within the subsystem. The facade should handle all the intricacies and dependencies of the subsystems.
- **Client Code Interaction:** The client interacts with the system through the facade, simplifying its use of the complex subsystems.

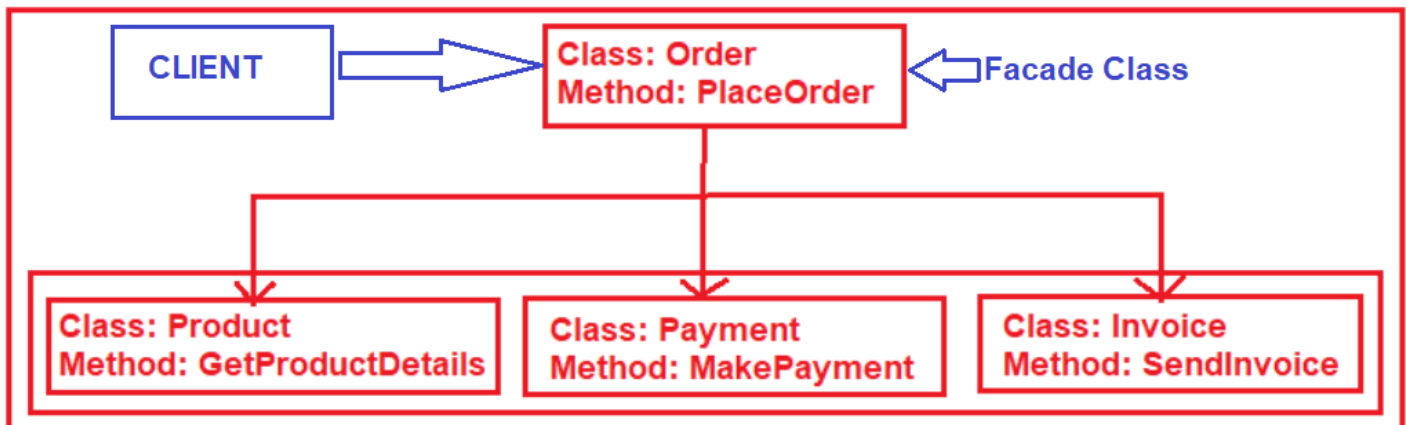
Let us understand the Facade Design Pattern in C# with one Real-Time Example. Please have a look at the following diagram for a better understanding. Here, we need to design an application to place an order in an E-Commerce Application.



As shown in the above image, to place an order first, the Client needs to create an object of the Product class and get the product details by calling the GetProductDetails method. Then, if everything is fine (i.e. if the Product is

available in stock), you need to make the Payment. To do this, the Client needs to create an instance of the Payment class and need to call the MakePayment method. If Payment is successful, then we need to send the Invoice to the customer and to do so, the Client needs to create an instance of the Invoice class and call the SendInvoice method. So, to place the order, the Client needs to do the above-mentioned steps in a particular order.

The Facade Design Pattern in C# is actually an extra class (i.e., a Wrapper class or, you can say, Facade Class) that sits at the top of the above classes. Please have a look at the following diagram for a better understanding.

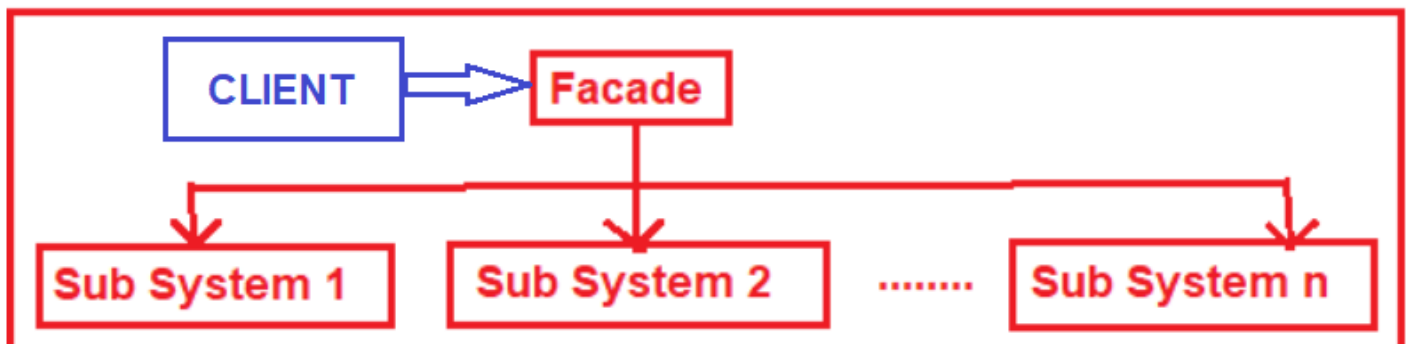


So, here, the extra class Order is nothing but the Facade class, which will be responsible for placing the order. This class internally creates the instance of the respective classes and calls the methods in a particular order. Now, the Client will not call the respective classes and their methods to place the order; instead, the Client will call the Order Class, PlaceOrder to method to place an order. The PlaceOrder method will internally use the Product, Payment, and Invoice classes to place the order.

Note: The point that you need to remember is the Facade Design Pattern not only decreases the overall complexity of the application but also helps to move the unwanted dependencies to one place. Facade deals with interfaces, not implementation. The actual implementation is going to be provided by the Subsystems.

Understanding the Class or UML Diagram of Facade Design Pattern:

Let us understand the class diagram (or UML Diagram) and the different components of the Facade Design Pattern. Please look at the following image to understand the Facade Design Pattern class diagram.



As shown in the above image, three classes are involved in the Facade Design Pattern. They are as follows:

1. The **Facade Class** knows which subsystem classes are responsible for a given request, and then it delegates the client requests to appropriate subsystem objects.
2. The **Subsystem Classes** implement their respective functionalities assigned to them, and these Subsystem Classes do not know the Facade class.
3. The **Client Class** uses the Façade Class to access the subsystems.

Note: We have used the Facade Design Pattern unknowingly so many times in our projects, even though we are not aware of this. If you are working on Web APIs, you are working with Facade Design Patterns. This is one of the most useful Design Patterns in Real-Time Applications. If you understand the Facade Design Pattern, you will improve your Project Architecture.

Implementing Facade Design Pattern in C#:

Let us implement the example we discussed step by step using the Facade Design Pattern in C#.

Step 1: Creating Subsystems

In our example, the Subsystem classes will be the Product, Payment, and Invoice classes, and each class will have its own responsibility. So, let's create the above three classes and implement their responsibility.

Product Subsystem:

Create a class file named **Product.cs**, then copy and paste the following code. This class has a method to get the product details. In a real-time application, you need to call this method to get the Product details like the Product Name, Product Images, Actual Price, Discounted Price, Quantity Available, etc. The following Product class can accept the request from the Facade or the client directly.

```
using System;
namespace FacadeDesignPattern
{
    // Subsystem 1
    // The Subsystem can accept requests either from the facade or from the
    client directly.
    // In this case, from the Subsystem, the Facade is also a client
    // Facade is not a part of the Subsystem.
    public class Product
    {
        public void GetProductDetails()
        {
            Console.WriteLine("Fetching the Product Details");
        }
    }
}
```

Payment Subsystem:

Once you make sure the Product is available, you need to do the Payment, and for this, we will use the Payment Subsystem. So, create a class file with the name **Payment.cs** and copy and paste the following code into it. This class has a method to do the payment. Here, you need to check how the customer will do the Payment. Whether Net Banking, UPI, or Cash on Delivery, you need to make the payment accordingly. The following Payment class can accept the request from the Facade class or the client directly.

```
using System;
namespace FacadeDesignPattern
{
    // Subsystem 2
    // The Subsystem can accept requests either from the facade or from the
    client directly.
    // In this case, from the Subsystem, the Facade is also a client
    // Facade is not a part of the Subsystem.
    public class Payment
    {
        public void MakePayment()
        {
            Console.WriteLine("Payment Done Successfully");
        }
    }
}
```

```

    }
}

```

Invoice Subsystem:

Once the Payment is Successful, we need to send the Payment Invoice to the customer, and for this, we will use the Invoice System. So, create a class file named **Invoice.cs** and copy and paste the following code. This class has a method called Sendinvoice to send the invoice. Sometimes, we are also sending the Payment Failed Invoice to the client. The following Invoice class can accept the request from the Facade class or the client directly.

```

using System;
namespace FacadeDesignPattern
{
    // Subsystem 3
    // The Subsystem can accept requests either from the facade or from the
client directly.
    // In this case, from the Subsystem, the Facade is also a client
    // Facade is not a part of the Subsystem.
    public class Invoice
    {
        public void Sendinvoice()
        {
            Console.WriteLine("Invoice Send Successfully");
        }
    }
}

```

Note: Here, we have not implemented the methods in detail except that we are just printing the details. This is because our idea is to understand the Facade Design Pattern implementation in C# and not to focus on the real implementations of the methods.

Step 2: Creating the Facade Class

This will be a concrete class, and this class takes the responsibility to place the order. It will work like a wrapper class. So, create a class file names **Order.cs** and copy and paste the following code. This class has one method that will create subclass objects and call the respective methods in a particular order to place an order. The following class provides a simple interface for the client to place an order. Now, the client will create an instance of the following Order class and needs to call the PlaceOrder method to place an order.

```

using System;
namespace FacadeDesignPattern
{
    // The Facade class provides a simple interface to the complex logic of
one
    // or several subsystems. The Facade delegates the client requests to
the
    // appropriate objects within the subsystem.
    public class Order
    {
        public void PlaceOrder()
        {
            Console.WriteLine("Place Order Started");
            //Get the Product Details
            Product product = new Product();
            product.GetProductDetails();
            //Make the Payment

```

```

        Payment payment = new Payment();
        payment.MakePayment();
        //Send the Invoice
        Invoice invoice = new Invoice();
        invoice.Sendinvoice();
        Console.WriteLine("Order Placed Successfully");
    }
}

```

Now, we are hiding the complexity of creating the different subclass objects and calling their respective methods with the help of the Facade class. So, this class acts as a wrapper for the subclasses. Now, the client will use this Facade class and call the PlaceOrder method to place an order. The PlaceOrder method takes all the responsibility for placing an order.

Step 3: Client

The class that is going to use the Facade class is nothing but the client. In our example, it will be the Main method of the Program class. So, please modify the Main method of the Program class as follows. Here, the client needs to create an object of the Order class and call the PlaceOrder method to place an order.

```

using System;
namespace FacadeDesignPattern
{
    class Program
    {
        static void Main(string[] args)
        {
            //The Client will use the Facade Interface instead of the
Subsystems
            Order order = new Order();
            order.PlaceOrder();
            Console.Read();
        }
    }
}

```

Output:

```

Place Order Started
Fetching the Product Details
Payment Done Successfully
Invoice Send Successfully
Order Placed Successfully

```

Advantages of Facade Design Pattern:

- **Simplified Interface:** Offers a single, simplified interface to the complex subsystems, making the subsystems easier to use.
- **Reduced Complexity:** Clients interact with a single unified interface rather than directly with the complex subsystems, reducing the system's perceived complexity.
- **Isolation:** Provides a degree of isolation from the complex subsystems, which can be beneficial when there are frequent subsystem changes.

- **Improved Testability and Maintainability:** Facade can simplify the testing process by limiting the interdependencies and focusing on system interfaces.

When to use Facade Design Patterns in Real-Time Applications?

The Facade Design Pattern is particularly beneficial in the following scenarios:

- **Simplifying Complex Systems:** When dealing with a complex system or framework with multiple interdependent classes or layers, you want to provide a simple interface to these systems. The Facade pattern can encapsulate this complexity behind a simple, unified interface.
- **Decoupling Systems:** If you want to decouple a system where the components are tightly coupled or interdependent, a facade can provide loose coupling by not exposing the internal complexities to the client.
- **Layered Architecture:** A facade can act as an entry point to each layer in multi-layered architecture. This is particularly useful in large applications or systems where each layer has complexities.
- **Improved Readability and Usability:** When you aim to improve the readability and usability of a system. A facade can provide a clear and straightforward way to interact with a complex subsystem, making it easier for other developers to understand and use it.
- **Reducing Dependencies:** To reduce external code dependencies on the inner workings of a library or framework, thereby shielding the client code from future changes or complexities in the subsystem.
- **Subsystem Interface Standardization:** For standardizing the interfaces of subsystems. If different subsystems have different interfaces, a facade can provide a uniform interface to all these subsystems, making them easier to work with.

In the next article, I will discuss [Real-Time Examples of Facade Design Patterns using C#](#). Here, in this article, I try to explain the **Facade Design Pattern in C#** with Examples. I hope you understand the need and use of Facade Design Patterns in C# with Examples.